MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

PROGRAM: M.Tech in CYBER SECURITY

SUBJECT: CSS5136 Information Security Mini Project

Smart Security Log Detection and Response

Report

Submitted by

K DHYANA SAMAGA

261091010031

Under the guidance of

GUIDE NAME

Mr.Manamohana K, Dr.Archana Praveen Kumar

Project repository:

https://github.com/KDhyanaSamaga/SentinelLog-AI

SCHOOL OF COMPUTER ENGINEERIN

Abstract

Modern networks generate security logs faster than any team can read them, and most of it is routine. Checking everything by hand doesn't scale, and running every log through an LLM is expensive overkill for traffic that's almost certainly benign. SentinelLog AI works on a simpler premise: not every log deserves the same attention. Machine learning triages incoming logs into Low, Medium, and High risk, based on pattern, frequency, severity, and system sensitivity. Only High-risk events reach an LLM, which explains the incident, flags the likely attack type, maps it to MITRE ATTCK, and suggests a response like blocking an IP. A dashboard ranks everything by priority.

Contents

Abstract **1**

1 Introduction **6**

1.1 Background 6

1.2 Motivation 6

1.3 Proposed System 6

1.4 Current Scope 7

1.5 Organisation of the Report 7

2 Literature Survey **8**

2.1 Existing Approaches 8

2.2 Comparison Survey Table 9

2.3 Outcome of the Literature Survey 10

2.4 Research Gaps 11

2.5 Problem Statement 11

2.6 Objectives 12

3 System Requirements and Architecture **13**

3.1 System Requirements 13

3.1.1 Hardware Requirements 13

3.1.2 Software Requirements 14

3.2 High-Level System Architecture 15

3.2.1 Architecture Overview 15

3.3 Functional Requirements 16

3.4 Non-Functional Requirements 17

3.5 Threat Model and Security Impact Analysis 18

3.5.1 Assets to be Protected 19

3.5.2 Identified Threats and Security Impact (Security Attacks) 19

3.5.3 Implemented Security Controls 20

4 Design Methodology **22**

4.1 Modular Architecture . . . 22

4.2 Data Flow Diagram . . . 22

4.3 Use Case Diagram . . . 23

4.4 Triage Decision Logic . . . 24

4.5 Database Design . . . 24

5 Implementation **25**

5.1 Implementation Workflow . . . 25

5.2 Tools and Libraries . . . 26

5.3 Dataset: Loghub HDFS . . . 26

5.4 Module Descriptions . . . 28

5.4.1 ML Model Module . . . 28

5.4.2 LLM Module . . . 28

5.4.3 Backend Module . . . 29

5.4.4 Database Module . . . 30

5.4.5 Frontend Module . . . 30

5.5 Setup and Execution . . . 30

6 Results and Analysis **33**

6.1 Model Training Pipeline . . . 33

6.2 Classification Results . . . 34

6.3 LLM Output Examples . . . 34

6.4 Application and Integration Results . . . 35

6.4.1 API and Server Verification . . . 36

6.4.2 Authentication, Persistence and Dashboard . . . 37

6.4.3 LLM Analysis and Final Dashboard . . . 39

6.5 Performance Analysis . . . 40

6.6 Limitations . . . 41

6.7 Future Scope . . . 42

7 Conclusion **43**

References **44**

List of Figures

3.1 High-level architecture of SentinelLog-AI 15

4.1 Level-1 data flow diagram of SentinelLog-AI 23
4.2 Use case diagram 23

5.1 Dataset summary 27
5.2 First three lines of the dataset 27
5.3 Log-level distribution 27
5.4 Graphical representation of log counts 28

6.1 Classification report output 35
6.2 LLM API testing result 35
6.3 FastAPI Swagger documentation: API endpoint verification, step 1 36
6.4 FastAPI Swagger documentation: API endpoint verification, step 2 36
6.5 FastAPI Swagger documentation: API endpoint verification, step 3 37
6.6 Uvicorn server running successfully 37
6.7 Initial SentinelLog-AI login interface 38
6.8 SentinelLog-AI dashboard after successful login 38
6.9 Persisted SentinelLog-AI data displayed in pgAdmin 38
6.10 Rate-limit error after more than five invalid login attempts 39
6.11 LLM-processed suspicious log displayed in the dashboard 39
6.12 Final dashboard showing log counts, warning and information totals, and reasons 40

List of Tables

2.1 Summary of literature survey, methodology and limitations 9

3.1 Suggested hardware requirements 13
3.2 Software requirements 14
3.3 Design target versus current implementation status 16
3.4 Functional requirements 16
3.5 Non-functional requirements 17
3.6 Assets to protect 19
3.7 Identified threats, attack scenarios, impact and mitigations 19

4.1 Backend modules 22

5.1 Libraries used (from `requirements.txt`) 26
5.2 HDFS_2k dataset summary 27
5.3 Main API endpoints 29

6.1 Classification report on the test set 34
6.2 Confusion matrix (rows: actual, columns: predicted) 34

1 Introduction

1.1 Background

Large volumes of security and framework logs are being created by modern information systems. These logs record system activity, departures, unusual behaviour and potential security threats, but analysing them manually is time-consuming and makes it hard for cybersecurity analysts to spot critical events quickly. LLMs can help explain security logs and potential threats. But still, analysing *every* log with an LLM augments computational demand, latency and operating cost, especially when most logs represent normal activity. Recent research has tried out language models and generative AI for log anomaly detection and for supporting security analysts, on the whole. And thus, LogFiT studies fine-tuned language models for spotting oddities in system logs [1], and work on generative AI for cybersecurity analysts studies how language models can support security analysis [2], to a significant degree. Retrieval-Augmented Generation (RAG) has been employed to analyse AWS CloudTrail events and map threats to the MITRE ATT&CK framework [3], and language models have been checked for threat detection in IoT security logs [4].

1.2 Motivation

Most logs are generally harmless. What's needed is a triage layer that's handling the bulk of routine traffic cheaply and reserves expensive, deeper reasoning for the few events that deserve it, in real-world scenarios. Alert fatigue for analysts and unnecessary spend on LLM inference is being caused by a framework that treats every entry as a potential incident, in many respects.

1.3 Proposed System

SentinelLog-AI is a multi-stage security log analysis system. In turn, routine events stay in the cheap path and are stored directly. Incoming logs are checked, structured and classified by a machine-learning model. Events classified as abnormal (or that the model can't classify

confidently) are forwarded to an LLM-based analysis service that:

- explains the incident in natural language,
- identifies the probable attack type,
- maps the behaviour to MITRE ATT&CK technique identifiers, and
- recommends a response action (for example, investigating or blocking an IP address).

Results are persisted in PostgreSQL and shown on a dashboard.

1.4 Current Scope

The implemented framework includes the FastAPI backend, database models and Alembic migrations, authentication and security controls, the ML training pipeline and inference service, the LLM analysis service, an asynchronous in-process ingestion queue, and a browser dashboard (HTML/CSS/JavaScript). Really, a Redis-backed worker layer, a centralised ML inference microservice, LangChain-based orchestration and a React/Electron dashboard are part of the longer-term design and aren't yet implemented (Table 3.3).

1.5 Organisation of the Report

Chapter 2 introduces the literature survey, research gaps, problem statement and targets. Chapter 4 is outlining the design methodology. Chapter 3 covers system requirements, architecture and the threat model. So, chapter 5 is explaining the setup and how to run it. Chapter 6 reports findings and investigation, shortcomings and future work. Chapter 7 is concluding.

2 Literature Survey

2.1 Existing Approaches

Existing research is exploring a few directions, including language-model-based log anomaly detection, generative AI for security analysis, retrieval-augmented threat detection and language models for IoT security logs, in real-world scenarios. And thus, four representative studies are, more accurately, summarised below.

1. LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models (2024) [1]. LogFiT fine-tunes a pretrained BERT-based language model employing self-supervised masked prediction on *normal* log data only, so it's needing no template vocabulary and no labelled oddities, to a significant degree. At inference, the model's top-k token prediction accuracy is used to decide whether new log data deviates from normal behaviour, in many respects. What it reports is F1 scores above baselines on the HDFS, BGL and Thunderbird datasets. SentinelLog-AI is sharing the goal of finding unusual behaviour in logs, but adds a risk-based escalation stage and an explanation/response stage.

2. Using Generative AI Models to Support Cybersecurity Analysts (2024) [2]. Balogh et al. study the black-box use of LLM chatbots as a support tool for security analysts and present case studies, covering spotting vulnerabilities in Android applications, on the whole. And thus, risks and issues of employing LLMs in security tasks are also tackled by them, in real-world scenarios. And what's more, even so, this motivates the explanation role of the LLM in SentinelLog-AI. Indeed, in the meantime, the cost of using an LLM indiscriminately motivates the ML pre-filter.

3. Retrieval-Augmented LLM for AWS Cloud Threat Detection: CloudTrail MITRE ATT&CK Mapping (2026) [3]. Adediran, Awuson-David and Ahmed evaluate a two-step RAG approach with Gemini 2.5 Pro on 200 CloudTrail events (122 malicious, 78 harmless) covering nine MITRE ATT&CK techniques, in many respects. Indeed, the RAG-enabled model hit an estimated 78% accuracy, 85% precision and 79% F1, at about 4.1 s and $0.00376 per event, on the whole. Seeing that every event passes through the LLM, cost and latency scale

with volume, and RAG-based enrichment is a possible future enhancement for SentinelLog-AI.

4. Evaluating Language Models for Threat Detection in IoT Security Logs (2025) [4]. Tejero-Fernández and Sánchez-Macían present a pipeline that's using fine-tuned LLMs for anomaly detection and mitigation recommendation on IoT security logs, on the whole. What's more, three open-source LLMs are checked, under the zero-shot, few-shot and fine-tuning methods, in many respects. Fine-tuned LLMs give better multi-class attack classification than the baselines, and detected threats are mapped to MITRE CAPEC with IoT-specific mitigation actions. And thus, standard ML classifiers are employed as baselines.

2.2 Comparison Survey Table

Table 2.1 compares the four reference studies with the proposed SentinelLog-AI approach.

Table 2.1: Summary of literature survey, methodology and limitations

Study	Methodology / Contribution	Relevance to SentinelLog-AI	Limitation / Opportunity
LogFiT (Almodovar et al., 2024) [1]	BERT-based model fine-tuned with self-supervised masked prediction on normal logs; top-k prediction accuracy used for anomaly decision; evaluated on HDFS, BGL, Thunderbird.	Research basis for applying language models to log anomaly detection.	Focused on anomaly detection only; does not explain incidents, map to ATT&CK or recommend responses. SentinelLog-AI adds ML triage before deeper LLM investigation.
Using Generative AI Models to Support Cybersecurity Analysts (Balogh et al., 2024) [2]	Black-box use of LLM chatbots as analyst support; case studies on security tasks such as Android vulnerability identification.	Supports using LLMs for incident explanation and analysis.	No automatic triage pipeline; indiscriminate LLM use raises cost. SentinelLog-AI limits the LLM to selected high-risk events.

Continued on next page

Table 2.1: Summary of literature survey, methodology and limitations (continued)

Study	Methodology / Contribution	Relevance to SentinelLog-AI	Limitation / Opportunity
Adediran et al., 2026 [3]	Two-step RAG with an LLM on CloudTrail events with ATT&CK mapping; 200-event evaluation set; reports accuracy, precision, F1, latency and cost per event.	Reference for ATT&CK mapping and contextual enrichment of LLM analysis.	Every event is processed by the LLM (cost per event); cloud-specific; small evaluation set. SentinelLog-AI uses staged ML-based selection; retrieval enrichment is future work.
Tejero-Fernández and Sánchez-Macían, 2025 [4]	Compares classical ML with zero-shot, few-shot and fine-tuned open-source LLMs on IoT logs; maps threats to MITRE CAPEC with mitigation actions.	Shows language models applied to security-log detection and recommendation.	LLMs applied to the data directly; IoT-specific. SentinelLog-AI allocates LLM effort by assessed risk.
Proposed SentinelLog-AI	ML classification (TF-IDF + temporal/frequency features + Linear SVM) followed by LLM analysis of selected events.	Combines risk triage, natural-language explanation, attack identification, ATT&CK mapping and response suggestions.	Needs systematic evaluation of detection performance, false positives, latency and LLM-usage reduction against an all-logs-to-LLM baseline.

2.3 Outcome of the Literature Survey

What the survey emphasizes is the potential of ML and LLMs for automated security-log assessment, to a significant degree. Research on generative AI for analysts is suggesting that LLMs can (to some degree) support incident analysis and the explanation of security information [2], in real-world scenarios. LogFiT is showing the relevance of language models for log anomaly detection [1], so research on IoT security logs is offering further context for checking language-model-based threat detection [4]. In fact, the value of including cybersecurity knowledge is, put simply, shown by the RAG-based AWS study when mapping events to MITRE ATT&CK [3]. The principal outcome is that combining automated risk prioritisation with

contextual LLM analysis is a pertinent approach for security-log detection and response, on the whole, and its efficacy must, though, be established experimentally by measuring detection performance, false-positive rate, processing time and the reduction in LLM usage compared with an method that's sending every log to an LLM, in actual application.

2.4 Research Gaps

1. **Risk-based selection of logs for LLM analysis.** Whether an ML selection stage can, to some extent, lower LLM usage while still catching critical events is an open question, in many respects; and thus, analysing every incoming log with an LLM increases cost and processing time.

2. **Integration of ML and LLM analysis.** Truly, the reviewed methods address anomaly detection, generative-AI-assisted investigation and LLM-based detection separately. The potency of the combined process must be checked with direct LLM analysis and ML-only category, in real-world scenarios.

3. **False positives and classification accuracy.** More accurately, normal events classified as high risk cause unnecessary LLM calls; suspicious events classified as low risk never receive deeper examination, to a significant degree; precision, recall, F1-score and confusion matrices are normally needed.

4. **Computational cost and efficiency.** Savings can't be assumed from the architecture alone; the number of logs forwarded to the LLM, token consumption, latency and running cost must be measured against a baseline that's sending all eligible logs to the LLM.

2.5 Problem Statement

Modern network environments are creating enormous volumes of security logs from servers, applications, network devices and other infrastructure, in real-world scenarios. Indeed, even as, these logs include beneficial indicators of threats, the majority represent routine activity, on the whole. Manually studying and connecting them is slow, increasingly impractical and causes alert fatigue, and static rule-based monitoring struggles with evolving attack patterns. Employing an LLM to every log introduces unnecessary computational cost and latency. There's therefore a

need for an intelligent, cost-efficient security-log examination framework that automatically is reading raw logs, assesses the risk of each event, prioritises events, reserves advanced inquiry for high-risk events, and explains them, spots probable attack types, maps behaviour to MITRE ATT&CK and recommends doable responses.

2.6 Objectives

1. Build a machine-learning classifier that sorts incoming logs into risk tiers based on pattern, frequency, severity and the importance of the source system, rather than treating every entry as a threat.

2. Parse and structure raw log data automatically so high volumes can be processed without manual formatting or cleanup.

3. Limit LLM involvement to high-risk events only, keeping routine and low-risk logs out of the expensive analysis path.

4. Use the LLM, when invoked, to explain flagged incidents in plain language, identify the probable attack type and map the behaviour to MITRE ATT&CK.

5. Produce response recommendations for high-risk incidents rather than raw alerts with no next step.

3 System Requirements and Architecture

3.1 System Requirements

SentinelLog-AI is employing a web frontend, a FastAPI backend, an ML classification component, a queue-based processing mechanism, an LLM analysis service and PostgreSQL for persistent storage, in real-world scenarios. Needs are, by design, divided into the hardware and software needs.

3.1.1 Hardware Requirements

The hardware needs depend on log volume and on whether the LLM is run locally or accessed remotely, to a significant degree. Table 3.1 lists suggested (not measured) specifications.

Table 3.1: Suggested hardware requirements

Component	**Suggested requirement**	**Purpose**
Processor	Modern multi-core processor, e.g. AMD Ryzen 5 or equivalent	Runs backend, ML pipeline, database and frontend
RAM	8 GB minimum for development; 16 GB or more recommended	Application execution, log processing, database operations
Storage	8 GB or more free space	Application files, dependencies, datasets, logs, database files
Operating system	Windows 11	Primary development environment
Network	Internet access for remotely hosted LLMs and package downloads	Model access, package installation, external services
GPU	Optional, depending on the model and inference environment	Can accelerate local LLM inference

3.1.2 Software Requirements

Table 3.2: Software requirements

Category	Software / technology	Purpose
Operating system	Windows 11	Development environment
IDE / editor	Visual Studio Code	Development and debugging
Language	Python	Backend, ML and LLM integration
Frontend	HTML, CSS, JavaScript	Dashboard structure, styling and API communication
Backend framework	FastAPI	REST API and component coordination
ASGI server	Uvicorn	Runs the FastAPI application
Feature extraction	TF-IDF, One-Hot Encoding, StandardScaler	Converts text, categorical and numeric fields into model features
ML algorithm	Linear SVM (`LinearSVC`)	Classifies logs
ML library	scikit-learn, pandas, NumPy, joblib	Pipeline, evaluation, model serialisation
LLM model	Qwen2.5-72B-Instruct	Explains high-risk logs
LLM access	Hugging Face Inference API (`huggingface_hub`)	Remote inference
Database	PostgreSQL	Logs, analysis results, incidents, users
Queue	Application-level asynchronous queue (`asyncio`)	Holds ingestion/analysis tasks without Redis
ORM / migrations	SQLAlchemy (async), Alembic	Database access and schema versioning
Security libraries	Argon2 (`argon2-cffi`), PyJWT, SlowAPI	Password hashing, tokens, rate limiting
API testing	Swagger UI / OpenAPI	Endpoint documentation and testing
Configuration	Environment variables, `.env`	Secrets and settings outside source code
CI security	GitHub Actions with Semgrep and Trivy	Static analysis and dependency scanning
Version control	Git, GitHub	Source control

3.2 High-Level System Architecture

3.2.1 Architecture Overview

The backend validates requests, coordinates log processing, is invoking the ML classifier, manages queued tasks, calls the LLM service when needed and stores findings in PostgreSQL, in real-world scenarios. And thus, the frontend communicates with the backend through REST APIs. Text from each log is converted to numerical features and passed with the other attributes to a Linear SVM, which anticipates the class, in actual application. Indeed, logs that are, to be sure, abnormal or unclassifiable are sent for deeper investigation to the LLM service. Task submission from task execution is being separated by the queue, so that ingestion requests are handled according to the queue's scheduling logic. The architecture is being shown by Figure 3.1.

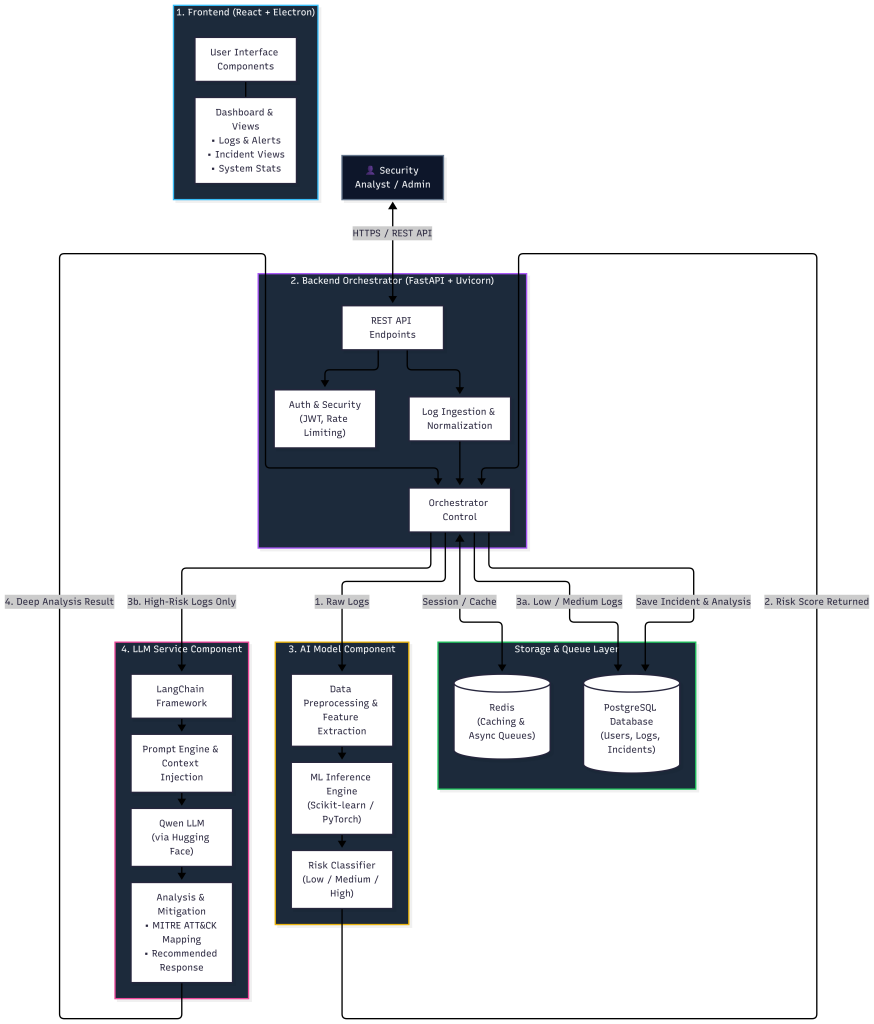


Figure 3.1: High-level architecture of SentinelLog-AI

Table 3.3: Design target versus current implementation status

Component	Design target	Current implementation
Risk classes	Low / Medium / High	Binary classifier (INFO vs. WARN) trained on HDFS; WARN is treated as high-risk and escalated
Queue	Redis-based workers	In-process asynchronous FIFO queue with ticket numbers and a status endpoint
LLM orchestration	LangChain	Direct call to the Hugging Face Inference API with a rule-based fallback
Frontend	React + Electron	HTML/CSS/JavaScript dashboard
ML serving	Centralised inference service	Model loaded in-process by the backend (`joblib`)

3.3 Functional Requirements

Table 3.4: Functional requirements

Requirement	Description
Log ingestion	Accept incoming security logs through the backend API (single and batch).
Parsing and preprocessing	Parse raw log messages, extract relevant fields and prepare text for classification.
Feature extraction	Transform log text and metadata into numerical feature vectors (TF-IDF, one-hot, scaled numerics).
Risk classification	Use the trained Linear SVM to classify incoming logs.
Risk-based routing	Route logs by class so that only high-risk or unclassifiable logs are submitted for LLM investigation.

Continued on next page

Table 3.4: Functional requirements (continued)

Requirement	Description
Task queue management	Place ingestion and analysis tasks in an application-level queue and expose queue status.
LLM-based investigation	Invoke the Qwen-based LLM service to investigate selected events.
Incident explanation	Generate human-readable explanations of the event and its implications.
Attack classification	Identify probable attack types and MITRE ATT&CK technique IDs.
Response recommendations	Suggest actions such as investigating suspicious IPs, reviewing services or considering IP blocking.
Persistent storage	Store logs, classification results, explanations and recommendations in PostgreSQL.
Authentication and tenancy	Register and authenticate organisations and admins; scope log data to the owning organisation.
Security dashboard	Display processed logs, classifications and incident information.
Incident prioritisation	Allow events to be viewed and prioritised by assigned risk.
Incident details	Provide original log data, classification and available investigation results for a selected event.
API documentation	Expose OpenAPI/Swagger documentation.

3.4 Non-Functional Requirements

Table 3.5: Non-functional requirements

Category	Requirement
Performance	Classify logs with minimal delay and avoid unnecessary LLM invocations for routine events.
Processing efficiency	Handle routine classification without an external LLM request per event.

Continued on next page

Table 3.5: Non-functional requirements (continued)

Category	Requirement
Scalability	Support future capacity growth through optimised preprocessing, inference and task management.
Reliability	Handle invalid inputs and processing failures without terminating the application; fall back gracefully when the LLM is unavailable.
Data persistence	Stored logs and results remain available in PostgreSQL after restarts.
Security	Protect log data and records against unauthorised access, disclosure and modification.
Input validation	Validate and sanitise all incoming log data and API parameters.
Maintainability	Modular architecture so classifier, LLM service and database layer can change independently.
Usability	Present events clearly so users can find high-priority incidents quickly.
Compatibility	Run in the supported Windows 11 development environment with compatible dependencies.
Observability	Keep operational logs and error information for debugging and monitoring.
Configuration management	Keep credentials and tokens outside source code.
Resource efficiency	Minimise CPU, memory, network and LLM inference use during routine processing.

3.5 Threat Model and Security Impact Analysis

SentinelLog-AI processes sensitive security information and interacts with several components: the backend API, an application programming interface, ML classifier, LLM analysis service, task queue and PostgreSQL. Confidentiality, integrity or availability may be changed by a weakness in any of them. The STRIDE methodology (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) [**?**] is employed to spot threats, assess their impact and choose mitigations.

3.5.1 Assets to be Protected

Table 3.6: Assets to protect

Asset	Description	Security importance
Raw security logs	Original messages from external sources	May contain sensitive system information, IP addresses, usernames and operational details
Classification results	ML-generated classes and scores	Incorrect modification may cause real threats to be overlooked
ML model and features	Trained pipeline (TF-IDF, encoders, Linear SVM)	Unauthorised modification affects accuracy and reliability
LLM analysis results	Explanations, attack types, ATT&CK mappings	May contain sensitive or misleading recommendations
Database records	Persisted logs, incidents, analyses	Must be protected from unauthorised reading, modification and deletion
API credentials	Database and LLM service credentials (`HF_TOKEN`)	Compromise gives access to connected services
Task queue	Pending investigation tasks	Tampering or exhaustion interrupts analysis
Application configuration	Environment variables and settings	Exposure may reveal credentials or internal details

3.5.2 Identified Threats and Security Impact (Security Attacks)

Table 3.7: Identified threats, attack scenarios, impact and mitigations

Threat	Attack scenario	Security effect	Suggested mitigation
Spoofing	Attacker submits fabricated logs impersonating a trusted source	Integrity: false events influence classification	Authenticate log sources; validate source identity
Log injection	Crafted content or control characters inserted in log messages	Integrity: records and downstream processing manipulated	Validate and normalise input; parameterised queries

Continued on next page

Table 3.7: Identified threats, attack scenarios, impact and mitigations (continued)

Threat	Attack scenario	Security effect	Suggested mitigation
ML evasion	Malicious patterns altered to resemble normal activity	Integrity: attacks classified as low risk	Evaluate adversarial inputs; monitor errors; retrain periodically
Queue exhaustion	Large volume of events generates many investigation tasks	Availability: queue and LLM service overloaded	Rate limiting; queue-size limits; resource controls
Prompt injection	Log text contains instructions aimed at the LLM	Integrity: misleading explanations or unsafe recommendations	Treat logs as untrusted data; isolate instructions from log content (delimiters); validate structured LLM output
Sensitive information disclosure	Credentials, tokens or personal data in raw logs forwarded to the LLM	Confidentiality: exposure to an external service	Redact sensitive fields; minimise transmitted data
Database injection	Malicious input aimed at database queries	Integrity and confidentiality: unauthorised access or modification	SQLAlchemy parameterised operations; validation; least-privilege DB user
Unauthorised API access	Access to ingestion or investigation endpoints without authorisation	Confidentiality and integrity: records exposed or altered	Authentication, authorisation and endpoint-level access control
Credential exposure	Secrets leaked through source code, logs or config files	Confidentiality: access to external services or storage	Environment variables or a secrets manager; never log credentials
Denial of service	Excessive API requests or oversized log messages	Availability: resources exhausted	Request-size limits, rate limiting, timeouts, monitoring
Repudiation	A user or attacker denies performing an operation	Accountability: forensic analysis becomes difficult	Timestamped audit records for important operations

3.5.3 Implemented Security Controls

The store writes several of these mitigations, including passwords are hashed with Argon2id (64 MiB memory cost, 3 versions, parallelism 4). Access tokens are signed JWTs (HS256) with a 15-minute lifetime and refresh tokens last 7 days. Login, registration and password-reset endpoints are rate restricted per IP address employing SlowAPI (to take one instance 5 login attempts per minute). Secrets are read from a `.env` file. Database access uses SQLAlchemy

parameterised queries. Log data is tied to an owning organisation. The LLM prompt wraps all log content in delimiters and instructs the model to treat it as untrusted data. And a GitHub Actions workflow runs Semgrep (OWASP Top Ten, Python, FastAPI rule sets) and a Trivy dependency scan on every push and pull request to `main`, in actual application.

4 Design Methodology

4.1 Modular Architecture

SentinelLog-AI follows a modular architecture in which each concern is separated in its own module with a stipulated interface, so that the classifier, LLM service or database layer can be changed without altering the others. The backend is organised as a set of feature modules, each with its own `api`, `schemas`, `service`, `repository` and `models` layers:

Table 4.1: Backend modules

Module	Route prefix	Responsibility
`auth`	`/api/v1/auth`	Registration, login, token refresh, sessions, permissions
`organization`	`/api/v1/organization`	Organisation profile and management
`log`	`/logs`	Ingestion, batch ingestion, queue, triage orchestration, explainable results
`aiModel`	`/model`	ML inference (`/predict`, `/health`)
`llmSservices`	`/llm`	LLM analysis (`/analyze`, `/health`) and prompt
`core`	—	Configuration, database engine, security primitives, rate limiter

The `LogTriageService` acts as the orchestrator that calls the ML service, decides whether to increase, calls the LLM service and carries on the finding, in many respects. In fact, within each module the layers are kept separate, namely the *API layer* handles HTTP, the *service layer* includes business logic, the *repository layer* is the only code that's touching the database, and *schemas* (Pydantic) validate data crossing a boundary, on the whole.

4.2 Data Flow Diagram

Figure 4.1 shows the level-1 data flow diagram. The dashboard is reading findings back through the API, an application programming interface. In fact, a log handed in by an confirmed real user is checked and queued, classified by the ML model, built up to the LLM only when needed, and stored, in real-world scenarios.

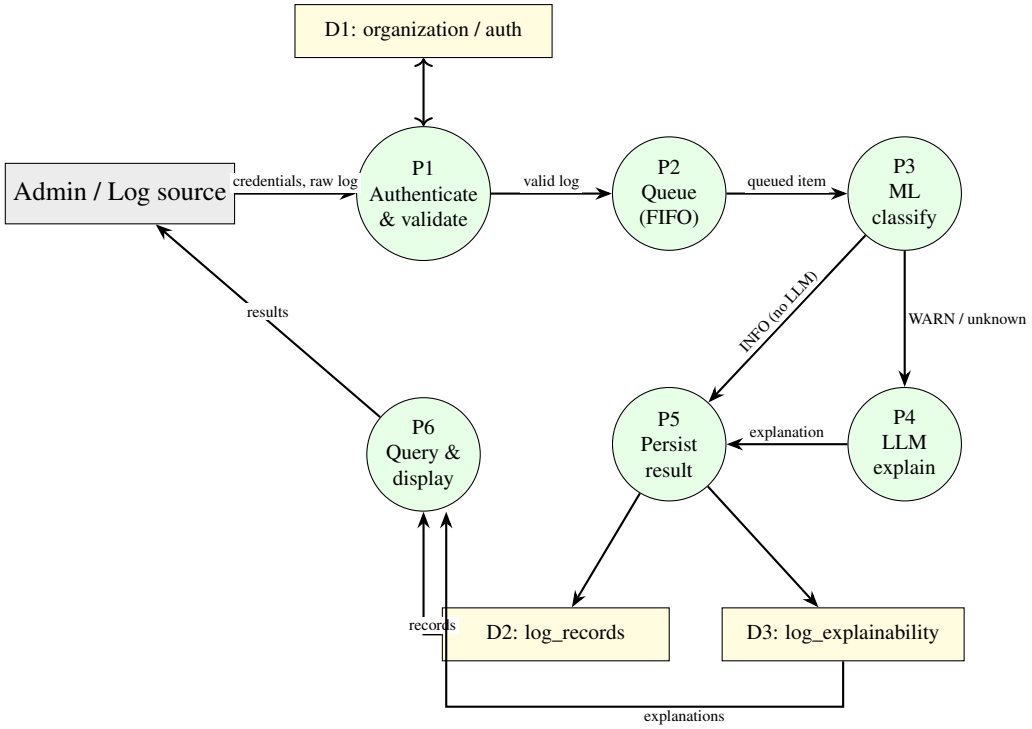


Figure 4.1: Level-1 data flow diagram of SentinelLog-AI

4.3 Use Case Diagram

Two actors interact with the framework, like the *Security Analyst / Admin*, who operates the dashboard, and the *Log Source*, an external system or script that turns in logs, in real-world scenarios. The LLM provider is an external service employed by the system. Figure 4.2 is showing the use cases.

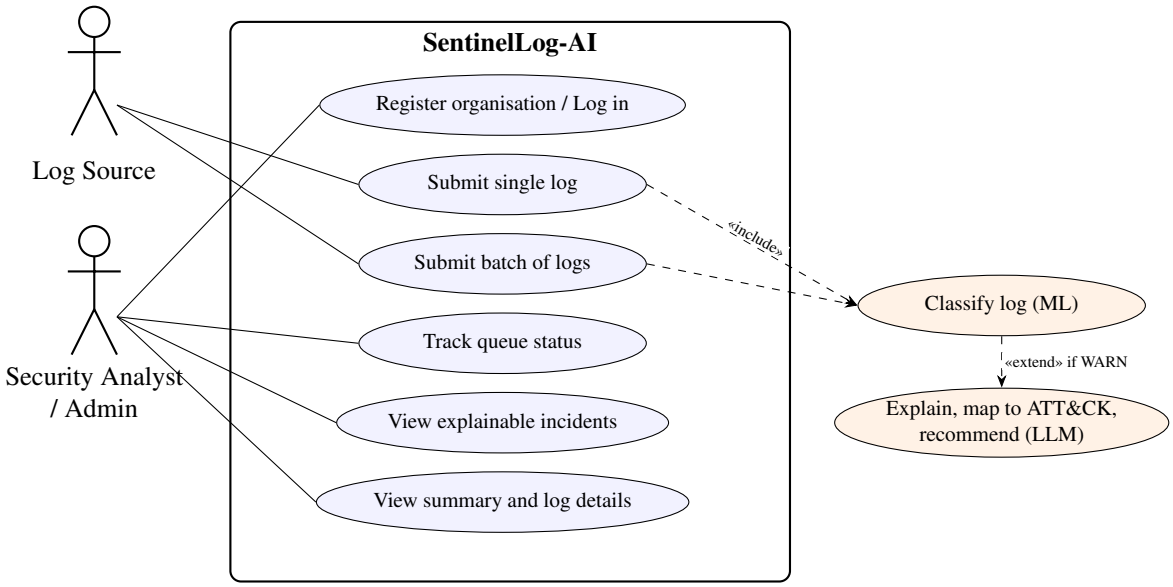


Figure 4.2: Use case diagram

4.4 Triage Decision Logic

The orchestrator increases a log to the LLM if any of the following holds, like the ML class is `WARN` (or `ERROR`/`UNKNOWN`). The model is often unavailable or couldn't classify the log. Or the log text comprises a high-severity keyword (for example *error*, *exception*, *corrupted*, *fatal*, *unauthorized*, *permission denied*, *checksum*, *fail*), on the whole. A keyword hit on an `INFO`-classified log is, to a large extent, pushing it to `WARN`. In fact, routine `INFO` logs are stored directly with `predicted_by = ML`, built up logs with `ML+LLM` (or `LLM` if the ML model couldn't project), in actual application.

4.5 Database Design

The PostgreSQL schema includes six tables: `organization` and `admin` (tenants and users), `auth_session` and `refresh_token` (session management), `log_records` (every ingested log with its traits, queue number, prediction and `predicted_by`) and `log_explainability` (LLM output linked to a log record), in actual application. Log rows carry an `organization_id` foreign key so that each organisation is seeing only its own data, to a significant degree. Schema changes are versioned with Alembic.

5 Implementation

5.1 Implementation Workflow

1. **Authenticate.** The user is logging in through the dashboard and receives a JWT access token, in many respects.

2. **Ingest.** The request is checked by Pydantic schemas and placed in the async FIFO queue; the caller receives a queue ticket number, in real-world scenarios; in fact, all i mean is it's, A log is turned in to `POST /logs/ingest` (or `/logs/batch`), broadly speaking.

3. **Classify.** A background worker is building the feature row (`Content`, `Component`, `Hour`, `Minute`, `Second`, `Time_gap`, `Pid_freq`) and runs the saved scikit-learn pipeline, to a significant degree.

4. **Route.** `INFO` `WARN`, unknown or keyword-flagged logs are generally increased. Logs without alarming keywords are, by design, stored directly.

5. **Explain.** A rule-based fallback is creating a heuristic explanation so that the pipeline doesn't fail, if the remote LLM is unavailable, on the whole; in fact, a guarded prompt is built by the LLM service, calls Qwen2.5-72B-Instruct through the Hugging Face Inference API, an application programming interface, and checks the structured JSON response, in actual application.

6. **Persist.** Plus The log record and (for increased logs) the explainability record are written to PostgreSQL, in actual application.

7. **Display.** The dashboard polls the queue status and reads summaries and explainable incidents, through the API, on the whole.

5.2 Tools and Libraries

Table 5.1: Libraries used (from `requirements.txt`)

Library	Use
`fastapi`, `uvicorn[standard]`	REST API framework and ASGI server
`sqlalchemy[asyncio]`, `asyncpg`, `psycopg[binary]`	Async ORM and PostgreSQL drivers (async at runtime, sync for migrations)
`alembic`	Database migrations
`pydantic-settings`, `python-dotenv`	Configuration from environment
`argon2-cffi`, `PyJWT`	Password hashing and token handling
`slowapi`	Rate limiting
`email-validator`, `python-multipart`	Input validation, form handling
`numpy`, `pandas`, `scikit-learn`, `torch`	Data processing and machine learning
`huggingface_hub`	Client for the LLM inference API

5.3 Dataset: Loghub HDFS

Loghub [5] is a large collection of system log datasets for AI-driven log analytics. In turn, the project employs the `HDFS_2k` sample, a 2,000-line structured subset stored at `backend/app/modules/aiModel/dataset/HDFS_2k.log_structured.csv`, in actual application. And thus, each row has the following fields: `LineId`, `Date`, `Time`, `Pid`, `Level`, `Component`, `Content`, `EventId` and `EventTemplate`, to a significant degree. The HDFS logs come from a Hadoop Distributed File System cluster [**?**].

The data is strongly imbalanced, so the classifier uses class weighting (`class_weight='balanced'`). `Date` and `Time` are encoded as HHMMSS integers, so `Time` is zero-padded to six digits and split into hour, minute and second.

Table 5.2: HDFS_2k dataset summary

Property	Value
Total log lines	2,000
`INFO` lines	1,920 (96%)
`WARN` lines	80 (4%)
Distinct components	6
Target column	`Level`
Train / test split	80% / 20% (1,600 / 400), stratified, `random_state=42`

```
Dataset loaded successfully.

1. Number of data rows: 2000
   Number of features (columns): 9

3. Features present in the dataset:
['LineId', 'Date', 'Time', 'Pid', 'Level', 'Component', 'Content', 'EventId', 'EventTemplate']

--- Dataset Summary Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   LineId         2000 non-null   int64
 1   Date           2000 non-null   int64
 2   Time           2000 non-null   int64
 3   Pid            2000 non-null   int64
 4   Level          2000 non-null   object
 5   Component      2000 non-null   object
 6   Content        2000 non-null   object
 7   EventId        2000 non-null   object
 8   EventTemplate  2000 non-null   object
dtypes: int64(4), object(5)
memory usage: 140.8+ KB
None
```

Figure 5.1: Dataset summary

```
--- First 3 Rows ---
```

	LineId	Date	Time	Pid	Level	Component	Content	EventId	EventTemplate
0	1	81109	203615	148	INFO	dfs.DataNode$PacketResponder	PacketResponder 1 for block blk_38865049064139...	E10	PacketResponder <*> for block blk_<*> terminating
1	2	81109	203807	222	INFO	dfs.DataNode$PacketResponder	PacketResponder 0 for block blk_-6952295868487...	E10	PacketResponder <*> for block blk_<*> terminating
2	3	81109	204005	35	INFO	dfs.FSNamesystem	BLOCK* NameSystem.addStoredBlock: blockMap upd...	E6	BLOCK* NameSystem.addStoredBlock: blockMap upd...

Figure 5.2: First three lines of the dataset

```
2. Log Level Distribution ('Level'):
Level
INFO    1920
WARN      80
Name: count, dtype: int64
/tmp/ipykernel_978/1698471442.py:36: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(data=df, x=col_name, palette='viridis')
```

Figure 5.3: Log-level distribution

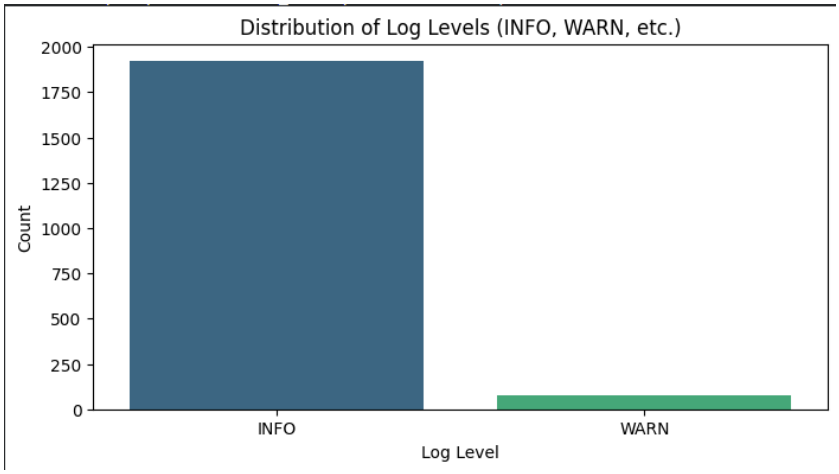


Figure 5.4: Graphical representation of log counts

5.4 Module Descriptions

5.4.1 ML Model Module

The ML module (`backend/app/modules/aiModel`) includes the training notebook (`main.ipynb`), the saved pipeline (`hdfs_log_classifier.joblib`), a Pydantic input/output schema, a service that loads the model once at start-up and an API (`POST /model/predict`, `GET /model/health`). The pipeline combines three feature groups in a `ColumnTransformer`:

- **Text:** TF-IDF on `Content` (maximum 1,500 features, English stop words removed);
- **Categorical:** one-hot encoding of `Component`;
- **Numeric:** standardised `Hour`, `Minute`, `Second`, `Time_gap` (seconds since the previous log) and `Pid_freq` (how often that process ID emerges), broadly speaking.

Bluntly, these are feeding a `LinearSVC` classifier [**?**, **?**]. At inference, the service reports the projected class, the decision-function margin. As, well, whether the log is needing LLM escalation, and returns `UNKNOWN` with escalation set, so the log is analysed by the LLM instead of being silently dropped, to a significant degree.

5.4.2 LLM Module

The LLM module (`backend/app/modules/llmSservices`) exposes `POST /llm/analyze` and `GET /llm/health`. And thus, the request includes `raw_log`, `log_type`, `machine` and `extra_context`. The prompt (`promt.py`) casts the model as a senior security analyst and wraps every input field in tags, telling the model never to follow instructions that are appearing inside the log data (a prompt-injection defence), on the whole. The model is Qwen2.5-72B-Instruct [6]

accessed through `InferenceClient`. The response is checked against a strict Pydantic schema, `ExplainabilityResult`, with the fields `threat_level` (LOW, MEDIUM, HIGH, CRITICAL or UNKNOWN), `detection_summary`, `explanation`, `recommended_action`, `attack_type`, `mitre_attack_ids` (at most 20), `llm_model` and `confidence_score`, in many respects. And thus, a rule-based fallback inspects the log for patterns such as permission denials, network timeouts, block errors and termination messages, and returns a finding in the same schema, if no `HF_TOKEN` is set up or the API stops working.

5.4.3 Backend Module

The backend (`backend/app/main.py`) creates the FastAPI application, registers CORS for the local frontend origins, attaches the rate limiter and includes the routers. A lifespan handler initialises and starts the queue manager at start-up and stops it on shutdown. The `log` module contains the `LogQueueManager` (an `asyncio` FIFO queue with configurable concurrent workers, ticket numbers and the states `QUEUED`, `PROCESSING`, `COMPLETED`, `FAILED`) and the `LogTriageService` that implements the decision logic of Section 4.4. Table 5.3 lists the main endpoints.

Table 5.3: Main API endpoints

Endpoint	**Purpose**
`GET /health`	Service health
`/api/v1/auth/*`	Registration, login, refresh, logout and related auth operations
`/api/v1/organization/*`	Organisation management
`POST /logs/ingest`	Submit one log; returns result or queue ticket
`POST /logs/batch`	Submit multiple logs
`GET /logs/queue/metrics`	Queue metrics
`GET /logs/queue/{queue_number}`	Status of a queued item
`GET /logs/summary`	Aggregate summary
`GET /logs/explainable`	List logs with LLM explanations
`GET /logs/explainable/{log_id}`	One explanation
`GET /logs/{log_id}`	One log record
`POST /model/predict, GET /model/health`	ML inference and health
`POST /llm/analyze, GET /llm/health`	LLM analysis and health

5.4.4 Database Module

PostgreSQL stores all persistent data through the async SQLAlchemy models. Six Alembic migration scripts under `backend/app/alembic/versions` create and evolve the schema, including tenant ownership for logs and the auth session and refresh-token tables. The `log_records` table is indexed on creation time, prediction and a unique queue number; `log_explainability` references the log record and holds the LLM output. Six tables are stated (Section 4.5).

5.4.5 Frontend Module

The frontend (`frontend/`) is a browser dashboard written in plain HTML, the standard web markup language, CSS, the web styling language, and JavaScript (`index.html`, `styles.css`, `app.js`), and is logging the user in through `/api/v1/auth/login`, stores the access token in the browser, and calls the backend with a Bearer token, which lets the user turn in logs (covering from a file), view the handled logs and explainable incidents, and see queue and summary information. A React/Electron dashboard is planned. The API base URL defaults to `http://localhost:8000` and can be changed in the interface.

5.5 Setup and Execution

The commands below assume Windows with Python 3.10+, PostgreSQL and Git installed. Use the equivalent activation command on Linux/macOS.

1. Clone the repository.

```
git clone https://github.com/KDhyanaSamaga/SentinelLog-AI.git
cd SentinelLog-AI
```

2. Create a virtual environment and install dependencies.

```
python -m venv venv
venv\Scripts\activate          # Linux/macOS: source venv/bin/activate
pip install -r requirements.txt
```

3. Create the PostgreSQL database.

```
psql -U postgres -c "CREATE DATABASE sentinellog;"
```

4. Configure environment variables. Copy the example file and edit it.

```
copy backend\.env.example backend\.env     # Linux/macOS: cp
```

```
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/sentinellog
SECRET_KEY=<generate a random string of at least 32 characters>
ALGORITHM=HS256
JWT_ISSUER=sentinellog-ai
JWT_AUDIENCE=sentinellog-ai
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
LOGIN_RATE_LIMIT=5/minute
HF_TOKEN=<your Hugging Face access token>
```

A strong secret can be generated with:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

5. Authenticate with Hugging Face (optional, enables the real LLM).

```
hf auth login
```

Without a token the system still runs, using the rule-based fallback explanation.

6. Apply database migrations.

```
cd backend
alembic -c app/alembic.ini upgrade head    # TODO: verify this path on your machine
```

7. Start the backend.

```
uvicorn app.main:app --reload
```

The API is served at `http://localhost:8000`; interactive documentation is at `/docs` (Swagger UI).

8. Start the frontend. The backend allows the origins `localhost:3000`, `localhost:5500` and `127.0.0.1:5500`. Serve the `frontend` folder on one of these, for example:

```
cd ..\frontend
python -m http.server 5500
```

Then open `http://localhost:5500`, register an organisation, log in and submit logs.

9. Test the pipeline. Example request body for `POST /llm/analyze`:

```
{
  "raw_log": "2026-09-17 09:14:22.401 [WARN] [AuthService] Failed login attempt from
     IP 192.168.1.105: User 'jdoe' reached max retry threshold (5/5). Account
    temporarily locked.",
  "log_type": "security_audit",
  "machine": "prod-auth-server-02",
  "extra_context": "User 'jdoe' belongs to the elevated administrator group.
    Consecutive failures originated from an internal subnet over a 2-minute period."
}
```

10. Retrain the ML model (optional). Open `backend/app/modules/aiModel/main.ipynb` in Jupyter or VS Code and run all cells. The last training cells save the pipeline as `hdfs_log_classifier.joblib` in the same folder.

6 Results and Analysis

6.1 Model Training Pipeline

The following code shows the feature engineering and model pipeline used for training. A small unused helper (`parse_time`) from the notebook has been omitted from the listing.

Listing 6.1: Feature engineering

```
# Split zero-padded HHMMSS time into parts
time_str = df['Time'].astype(str).str.zfill(6)
df['Hour']   = time_str.str[:2].astype(int)
df['Minute'] = time_str.str[2:4].astype(int)
df['Second'] = time_str.str[4:].astype(int)

# Timestamp in seconds, then gap between consecutive log entries
timestamp = df['Time'].apply(
    lambda x: int(str(x).zfill(6)[:2]) * 3600
            + int(str(x).zfill(6)[2:4]) * 60
            + int(str(x).zfill(6)[4:]))
df['Time_gap'] = timestamp.diff().fillna(0)

# PID frequency (process-level statistic)
pid_counts = df['Pid'].value_counts().to_dict()
df['Pid_freq'] = df['Pid'].map(pid_counts)

text_col   = 'Content'
cat_cols   = ['Component']
num_cols   = ['Hour', 'Minute', 'Second', 'Time_gap', 'Pid_freq']
target_col = 'Level'

X = df[[text_col] + cat_cols + num_cols]
y = df[target_col]
```

Listing 6.2: Preprocessing and model pipeline with class weighting

```
preprocessor = ColumnTransformer(transformers=[
```

```
    ('text', TfidfVectorizer(max_features=1500, stop_words='english'), text_col),
    ('cat',  OneHotEncoder(handle_unknown='ignore'), cat_cols),
    ('num',  StandardScaler(), num_cols)
])

model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LinearSVC(class_weight='balanced',
                             random_state=42, max_iter=10000))
])
```

6.2 Classification Results

The model was evaluated on a stratified 20% hold-out set of 400 logs (384 `INFO`, 16 `WARN`).

Table 6.1: Classification report on the test set

Class	Precision	Recall	F1-score	Support
INFO	1.00	1.00	1.00	384
WARN	1.00	1.00	1.00	16
Accuracy			1.00	400
Macro average	1.00	1.00	1.00	400
Weighted average	1.00	1.00	1.00	400

Table 6.2: Confusion matrix (rows: actual, columns: predicted)

	Pred. INFO	Pred. WARN
Actual INFO	384	0
Actual WARN	0	16

All 400 test logs were classified correctly: there were no false positives (no normal log was escalated unnecessarily) and no false negatives (no `WARN` log was missed).

6.3 LLM Output Examples

Two manual tests of the LLM analysis service were recorded in the project documentation, in real-world scenarios. Indeed, for a failed-login log (five consecutive failures on an administrator account from an internal tackle), the service flagged a possible brute-force or unauthorised-access try and recommended immediate study, on the whole. For a routine HDFS `PacketResponder`

```
Classification Report:
              precision    recall  f1-score   support

        INFO       1.00      1.00      1.00       384
        WARN       1.00      1.00      1.00        16

    accuracy                           1.00       400
   macro avg       1.00      1.00      1.00       400
weighted avg       1.00      1.00      1.00       400

Confusion Matrix:
[[384   0]
 [  0  16]]
```

Figure 6.1: Classification report output

. . . `terminating` log, it correctly sketched the entry as normal operational behaviour with no security concern. This shows the intended behaviour, such as the LLM usefully is explaining for abnormal events and doesn't over-alarm on routine ones. These are qualitative checks and not a systematic check.

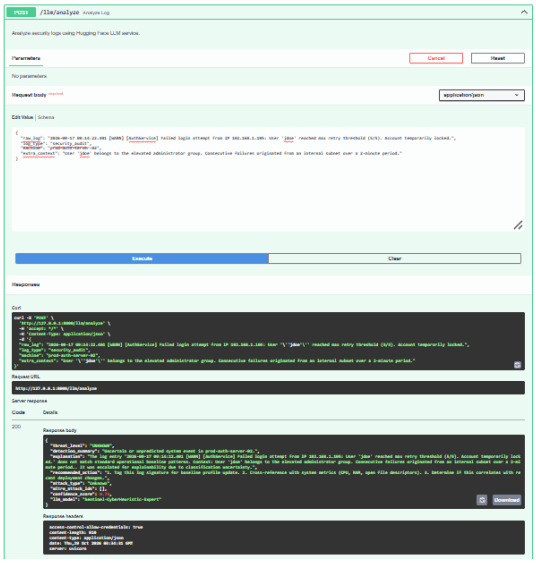


Figure 6.2: LLM API testing result

6.4 Application and Integration Results

The following screenshots document the system from startup through authenticated use, API verification, persistence and final log analysis. Together, they provide visual evidence that the implemented components operate as an integrated application.

6.4.1 API and Server Verification

The FastAPI Swagger interface was used to verify the available endpoints in sequence. The first three screenshots show the API documentation and endpoint testing workflow.

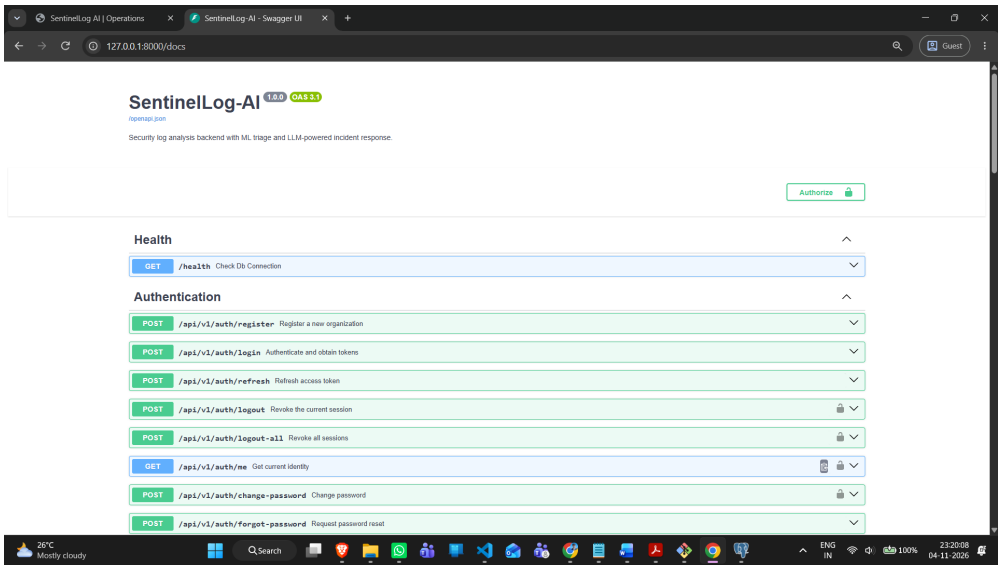


Figure 6.3: FastAPI Swagger documentation: API endpoint verification, step 1

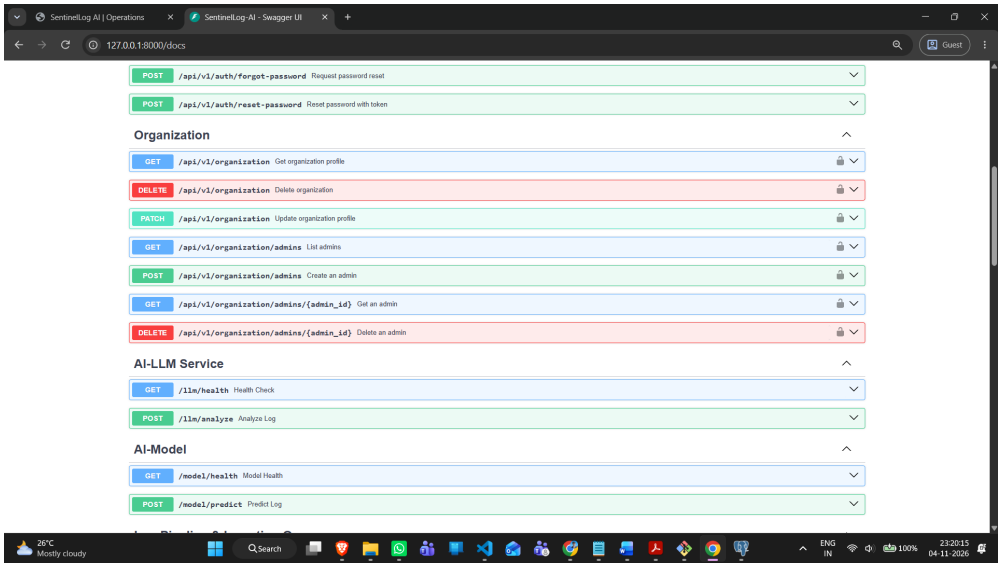


Figure 6.4: FastAPI Swagger documentation: API endpoint verification, step 2

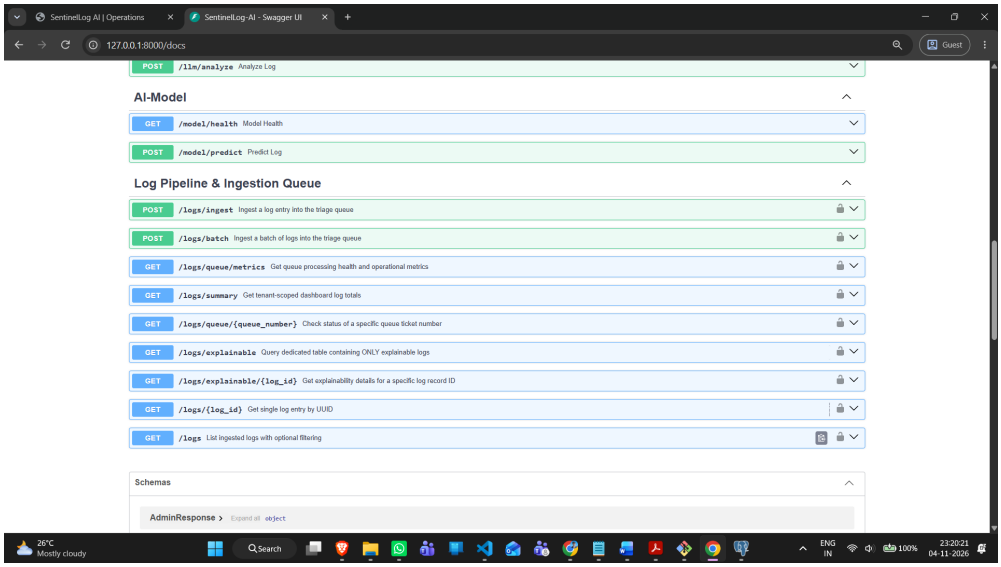


Figure 6.5: FastAPI Swagger documentation: API endpoint verification, step 3

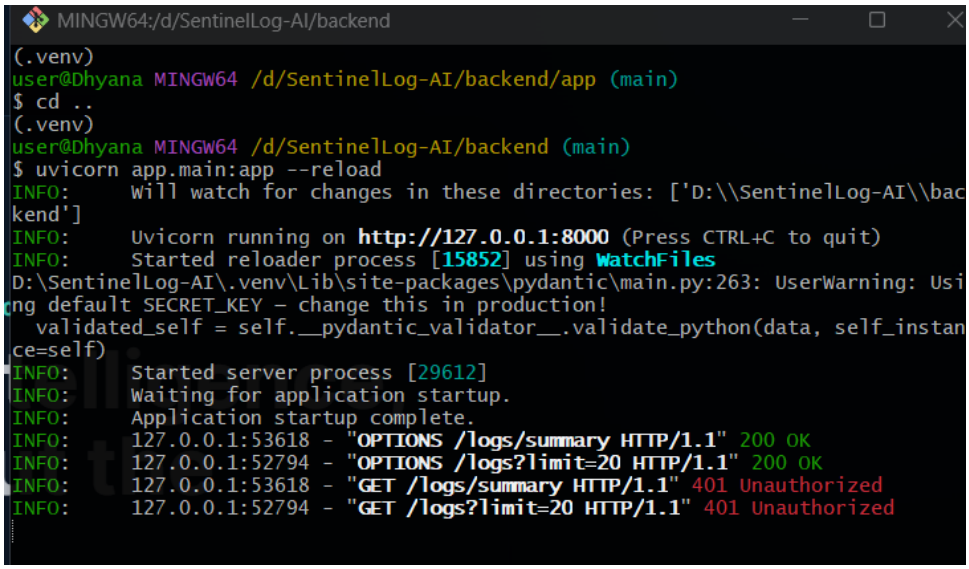


Figure 6.6: Uvicorn server running successfully

6.4.2 Authentication, Persistence and Dashboard

The application interface was then tested through the browser. The screenshots show the initial login screen, the dashboard after authentication, PostgreSQL records viewed in pgAdmin and the rate-limiting response after more than five invalid login attempts.

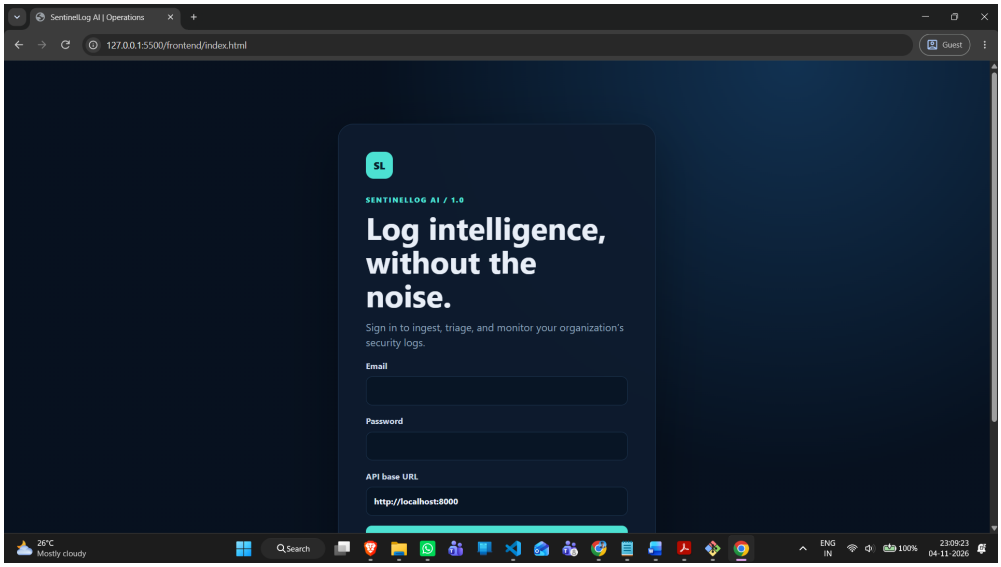


Figure 6.7: Initial SentinelLog-AI login interface

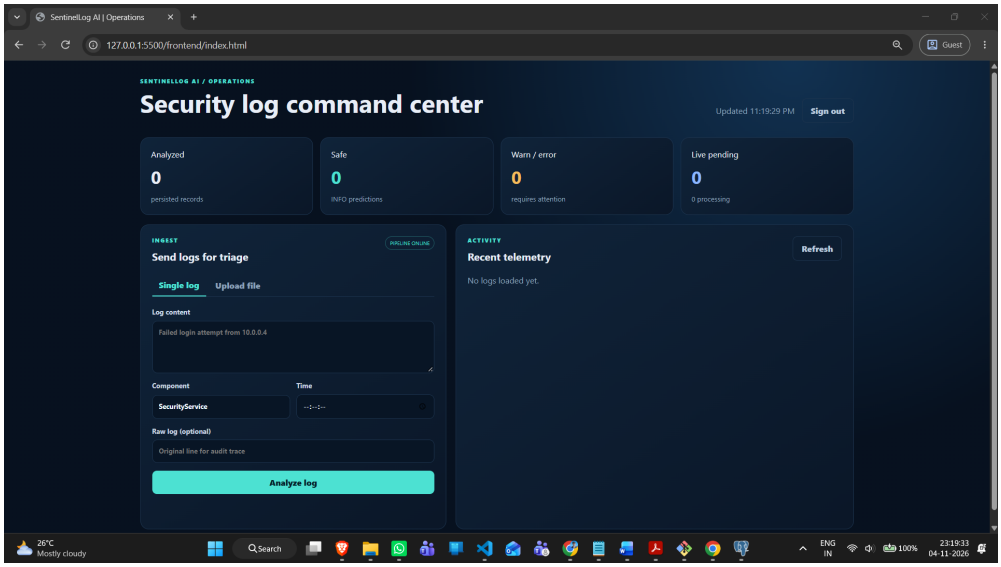


Figure 6.8: SentinelLog-AI dashboard after successful login

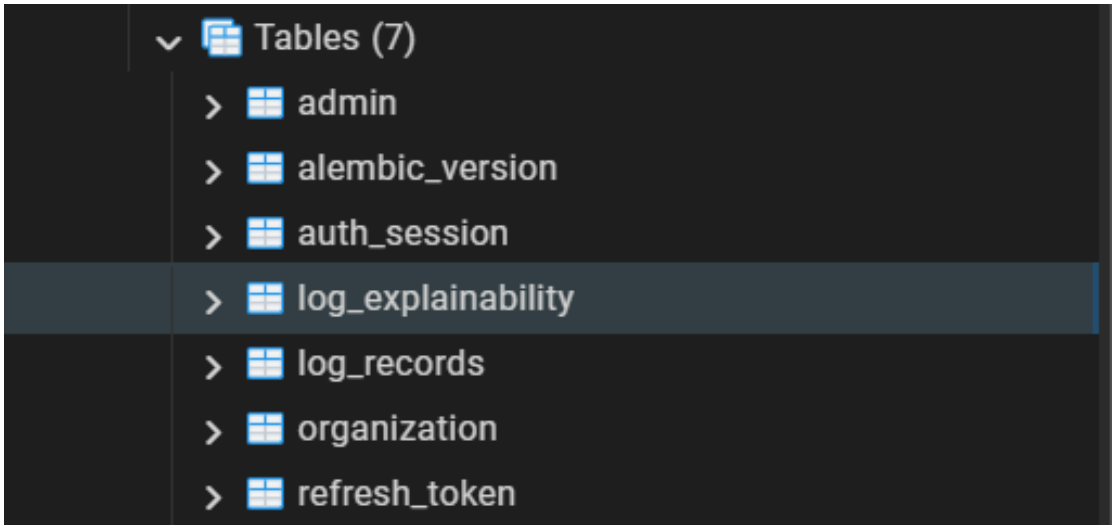


Figure 6.9: Persisted SentinelLog-AI data displayed in pgAdmin

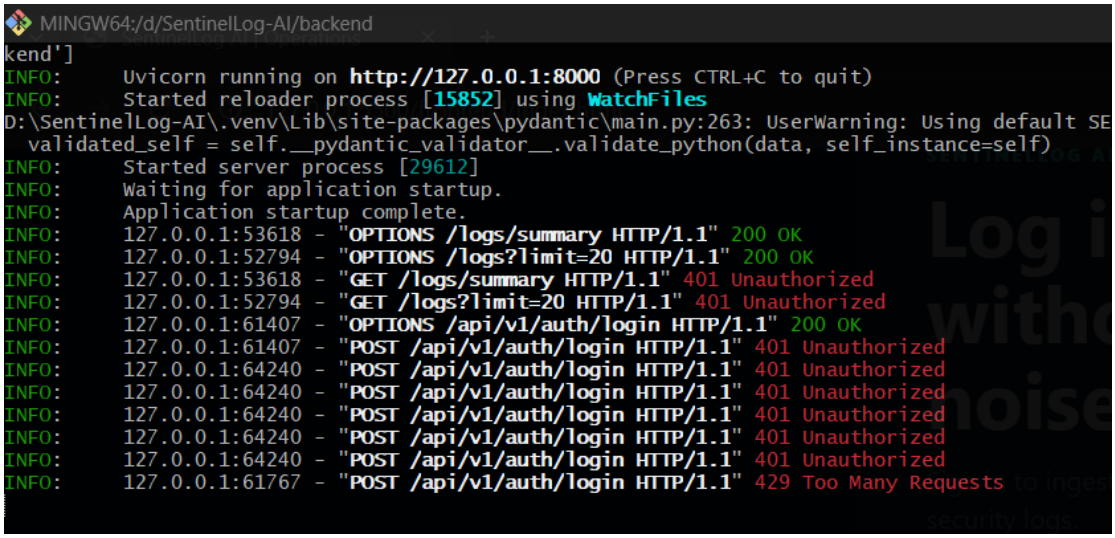


Figure 6.10: Rate-limit error after more than five invalid login attempts

6.4.3 LLM Analysis and Final Dashboard

Suspicious logs are escalated for LLM analysis and the resulting explanation is shown in the dashboard. The final dashboard aggregates the processed logs by level and displays the associated reasons.

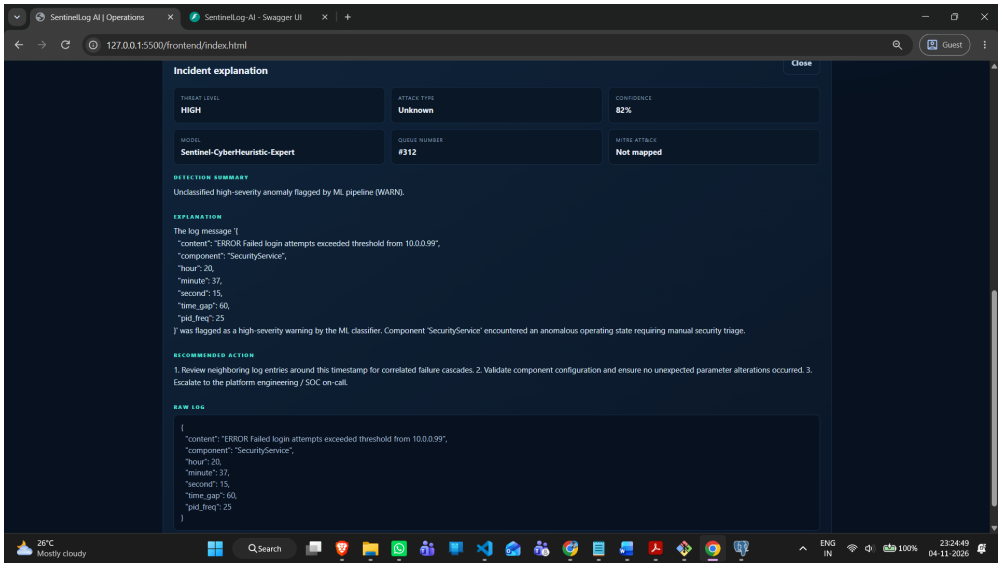


Figure 6.11: LLM-processed suspicious log displayed in the dashboard

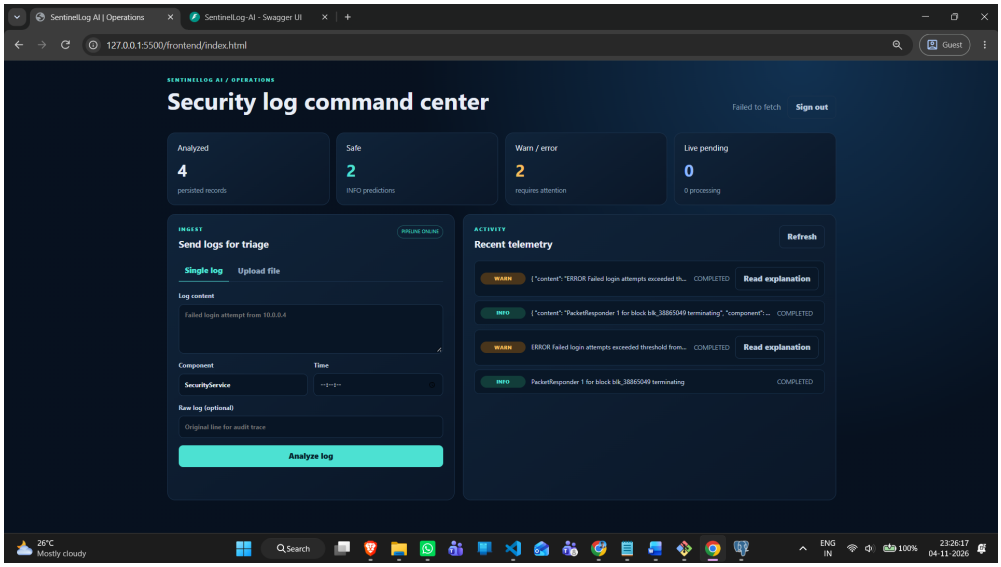


Figure 6.12: Final dashboard showing log counts, warning and information totals, and reasons

6.5 Performance Analysis

Strengths.

- Perfect precision and recall on both classes means the classifier would keep routine logs out of the LLM path and wouldn't miss the abnormal ones in this dataset, which is the behaviour the architecture rests on, in real-world scenarios. Inference on a single log is cheap, so the classifier can (to some degree) triage every log without the cost of an LLM call, in many respects.
- `LinearSVC` on thin TF-IDF features is very fast to train and to run, in actual application. The 96%/4% imbalance is being handled by
- Class weighting. Without it, a model that always projects `INFO` would already reach 96% precision while missing every warning, on the whole.
- The decision logic is currently conservative, like unknown or unclassifiable logs and keyword hits are increased rather, broadly speaking. Still ignored. An LLM outage is, indeed, degrading to a rule-based explanation rather than a failure.

Why 100% should be read with caution.

- **Small, easy test set.** The test set includes only 16 `WARN` logs; and thus, HDFS log levels are tied closely to a small number of message templates, so the label is largely figured out by the message text and component, which is making the task easy for a linear model, in

actual application.

- **Possible information leakage.** `Time_gap` and `Pid_freq` are computed over the whole dataset before the train/test split, so test rows impact the attributes of training rows, on the whole. This is mild, but a stricter protocol would compute them, within each split or in a streaming fashion, in real-world scenarios.
- **A different task from threat detection.** As, well, the target is the original log *level* (`INFO`/`WARN`), not a security-risk label. High precision is showing the model has learned to reproduce log severity, not that it can detect attacks.
- **Single small sample.** The 2,000-line sample comes from one framework. Findings may (to a meaningful extent) not carry over to the full HDFS dataset or to other log sources, broadly speaking.

A fair claim is therefore that the pipeline works end to end and the classifier is reliable on this dataset. It does not yet show generalisation.

6.6 Limitations

1. The classifier is binary (INFO/WARN) and trained on one HDFS sample. The Low/Medium/High tiers recapped in the design aren't yet made, on the whole.

2. No labelled attack data was, in effect, employed, so detection of real intrusions is normally untested, to a significant degree.

3. The framework's central claim (lessening LLM usage without missing critical events) hasn't yet been measured against an all-logs-to-LLM baseline for token use and latency, in many respects. Pending tasks are lost if the server restarts.

4. The queue is in-process: it isn't durable, has no automatic retry and doesn't scale, across the processes, in actual application.

5. LLM output isn't independently checked. Hallucinated MITRE IDs or unsafe recommendations are possible, and calling an external API means log content leaves the framework unless it's redacted, in real-world scenarios.

6. The ML model is usually vulnerable to evasion by attackers who craft log text to resemble normal entries, and isn't retrained automatically as log patterns drift, broadly speaking.

7. The model file is stored in the repo, and also, loaded without integrity checking, so this is a tampering risk in production (see the threat model), in real-world scenarios.

8. The frontend is a elementary dashboard. Role-based views and the MITRE ATT&CK visualisation outlined in the design aren't yet implemented, in real-world scenarios.

6.7 Future Scope

- As, well, train on the full labelled Loghub HDFS and BGL datasets and add attack-oriented datasets; appraise with cross-validation and time-based splits, and report precision, recall, F1 and ROC/PR curves, in actual application.
- And generalize to three risk tiers (Low, Medium, High) employing severity, frequency and system criticality, possibly with adjusted probabilities.
- Measure LLM-call reduction, token consumption and latency against a baseline that's sending all logs to the LLM, to a significant degree.
- What's more, replace the in-process queue with Redis Streams (or a akin broker) with durable storage, retries and separate workers, as in the target design, in real-world scenarios.
- Add RAG over the MITRE ATT&CK knowledge base to ground technique mapping and reduce hallucinated IDs [3], broadly speaking.
- Add LangChain orchestration, redaction of sensitive fields before LLM calls, and a local LLM option for privacy-sensitive deployments, in many respects.
- Build the React/Electron dashboard with incident review, MITRE ATT&CK views and analyst approve/reject workflow with audit logging, on the whole.
- Add drift monitoring and scheduled retraining, and adversarial testing of the classifier.

7 Conclusion

A applied, cost-aware way is shown by SentinelLog-AI to combine machine learning and LLMs for security-log examination, broadly speaking. A fast TF-IDF and Linear SVM classifier handles the full log stream. What's more, only abnormal or uncertain events are sent to an LLM for explanation, attack identification, MITRE ATT&CK mapping and response suggestions, in many respects. The implemented framework includes a secured FastAPI backend with PostgreSQL storage, an async ingestion queue, an ML triage service, an LLM analysis service with a tough fallback and a working dashboard. On the HDFS_2k sample the classifier hit perfect scores on a 400-log hold-out set, but still, the small and easy nature of the data means this should be treated as a proof of notion rather than indication of general performance. Broader datasets, three-tier risk classification, quantified cost savings and a durable queue are the next phases towards a production-ready system.

Bibliography

[1] C. Almodovar, F. Sabrina, S. Karimi, and S. Azad, "LogFiT: Log anomaly detection using fine-tuned language models," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 1715–1723, 2024. doi:10.1109/TNSM.2024.3358730.

[2] Š. Balogh, M. Mlynček, O. Vraňák, and P. Zajac, "Using generative AI models to support cybersecurity analysts," *Electronics*, vol. 13, no. 23, Art. 4718, 2024. doi:10.3390/electronics13234718.

[3] G. Adediran, K. Awuson-David, and Y. Ahmed, "Retrieval-augmented large language model for AWS cloud threat detection and modelling: CloudTrail MITRE ATT&CK mapping," *Computers, Materials & Continua*, vol. 87, no. 2, p. 100, 2026. doi:10.32604/cmc.2026.077606.

[4] J. J. Tejero-Fernández and A. Sánchez-Macían, "Evaluating language models for threat detection in IoT security logs," arXiv:2507.02390 [cs.CR], Jul. 2025. doi:10.48550/arXiv.2507.02390.

[5] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2023. Dataset repository: https://github.com/logpai/loghub.

[6] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[7] S. Ramírez, "FastAPI documentation." https://fastapi.tiangolo.com/.